



WIGMS

Wedding & Invitation Guest Management System

Date: May 2026

MEMBERS	PERCENTAGE OF PARTICIPATION
TIOFACK KAZE FRANCK	25%
PETTANG FEUMBA RODDY ALANN	25%
DJATCHIE DOMFANG STEEVIE RAZIEL	16.66%
MEP MEKU LARISSA AUDREY	16.66%
TAMO TABOUM ISRAEL	16.66%

1. Executive Summary

The Wedding and Invitation Guest Management System (WIGMS) is a comprehensive C-language application developed as part of a Data Structures and Programming course. The system is designed to digitally manage every aspect of a wedding event — from tracking guests and categories, to managing parking assignments, gifts, reports, and a graphical user interface via GTK. The project is hosted collaboratively on GitHub under the repository [franckkaze4-collab/WIGMS](#) and represents a modular, production-quality C codebase.

WIGMS integrates multiple advanced programming concepts, including dynamic memory allocation via singly linked lists, binary file I/O for persistent storage, CSV export functionality, full CRUD operations on structured data, and a GTK-based graphical interface. The system is organized into clearly separated modules — Person, Category, Gifts, Parking, Report, and GTK — each addressing a specific domain of the wedding management workflow.

Key Highlights

6 functional modules | Singly linked list architecture | Binary file persistence | CSV export | GTK GUI | Full CRUD on all entities | GitHub collaborative development

2. Introduction

2.1 Project Background

Wedding and event management involves handling large amounts of structured data — guest lists, seating categories, vehicle parking slots, gift records, and generated reports. Managing this manually with paper lists or simple spreadsheets is error-prone and difficult to maintain. WIGMS was designed to solve this problem programmatically, applying foundational computer science principles to a real-world scenario.

This project was developed as part of a university-level Data Structures and Programming course, with the goal of demonstrating mastery over C programming, dynamic data structures, file I/O, and software design. The project is maintained collaboratively on GitHub, simulating a real software development workflow.

2.2 Problem Statement

Wedding organizers face challenges in efficiently managing: (a) categorized guest lists with personal details, (b) parking assignments linked to guests or vehicles, (c) gift registries tied to specific guests, and (d) dynamically generated summary reports. Without a centralized digital system, these tasks are fragmented and difficult to update in real time.

2.3 Project Objectives

- Design and implement a modular C application to manage wedding guest data
- Use singly linked lists to enable dynamic memory management
- Implement full CRUD operations across all modules
- Provide binary file-based data persistence across sessions
- Support CSV data export for reporting and interoperability
- Develop a GTK-based graphical user interface
- Maintain the codebase on GitHub with collaborative practices

2.4 Scope

The scope of WIGMS covers six functional domains: Person management, Category management, Gift tracking, Parking assignment, Report generation, and a GTK GUI front-end. It does not include network connectivity, a web back-end, or database integration — all data persistence is achieved through binary file I/O.

3. GitHub Repository Structure

3.1 Repository Overview

The WIGMS project is hosted at <https://github.com/franckkaze4-collab/WIGMS>. The repository follows a clean module-based directory layout, making it easy for collaborators to navigate, contribute, and review. The repository includes all source files (.c, .h), Makefiles for compilation, and supporting documentation such as README files.

3.2 Repository Layout

```
WIGMS/
├── README.md      # Project overview and instructions
├── Makefile       # Root build script
├── main.c         # Application entry point
├── Person/
│   ├── person.h   # Person struct and function declarations
│   └── person.c   # Person CRUD implementation
├── Category/
│   ├── category.h # Category struct and declarations
│   └── category.c # Category linked list implementation
├── Gifts/
│   ├── gift.h     # Gift struct and declarations
│   └── gift.c     # Gift CRUD, file I/O, CSV export
├── Parking/
│   ├── parking.h  # Parking struct and declarations
│   └── parking.c  # Parking module implementation
├── Report/
│   ├── report.h   # Report declarations
│   └── report.c   # Report generation logic
├── GTK/
│   ├── gtk_ui.h   # GTK UI declarations
│   └── gtk_ui.c   # GTK graphical interface
```

3.3 Collaborative Development

The repository was set up with multiple collaborators, following standard GitHub workflows. The main branch holds production-ready code. Collaborators were added via GitHub's repository settings, enabling push access for team members. Files were managed both through the GitHub web interface and via command-line Git operations (git add, git commit, git push).

Practice	Tool / Command	Purpose
Version Control	git commit / git push	Track changes and sync to remote
Collaboration	GitHub Collaborators	Grant push access to teammates
Documentation	README.md files	Explain modules and usage

Practice	Tool / Command	Purpose
Build System	Makefile	Compile multi-module C project
Branch Management	main branch	Stable production codebase

4. System Architecture

4.1 Architectural Overview

WIGMS follows a modular, procedural architecture in C. Each domain of the system — persons, categories, gifts, parking, and reports — is encapsulated in its own module comprising a header file (.h) and an implementation file (.c). The main.c file serves as the program entry point, coordinating module interactions through a menu-driven command-line interface, supplemented by a GTK graphical interface.

4.2 Core Data Structures

4.2.1 Person Structure

The Person struct is the fundamental data unit. It represents a guest and is embedded inside Category nodes. Each person holds personal information including name, age, contact details, and unique identifier.

```
typedef struct Person {
    int id;
    char name[100];
    int age;
    char email[100];
    char phone[20];
} Person;
```

4.2.2 Category Structure (Linked List Node)

The Category struct forms the nodes of the singly linked list. Each category stores up to 4 guests (via a static array of Person), a unique ID, a code/label, and a pointer to the next node. This design enables $O(1)$ insertion at the head and $O(n)$ traversal for search, deletion, and update operations.

```
typedef struct Category {
    int id;
    char code[50];
    Person guests[4];
    struct Category *next;
} Category;
```

4.2.3 Gift Structure

The Gift module uses its own linked list of Gift nodes. Each node stores the gift ID, description, associated guest ID, monetary value, and a pointer to the next gift.

```
typedef struct Gift {
    int id;
```

```

char description[200];
int guest_id;
float value;
struct Gift *next;
} Gift;

```

4.2.4 Parking Structure

The Parking module also uses a linked list. Each node records a parking slot number, vehicle plate, assigned guest ID, and availability status.

```

typedef struct Parking {
    int slot_id;
    char plate[20];
    int guest_id;
    int available;
    struct Parking *next;
} Parking;

```

4.3 UML Class Diagram Summary

The class diagram (as shown in the lab document) depicts the following key relationships:

- Person class: attributes name, age, email, phone; constructors Person() and Person(name, age, email, phone)
- Category class: attributes id, code, guests[4], next; operations for full CRUD, count, sort, display
- Composition: One Category contains 0..4 Person instances (composition relationship)
- Self-reference: Category.next is a pointer to another Category (linked list)

4.4 Module Interaction Diagram

```

main.c
├── Category Module — uses — Person Module
├── Gift Module — references — Person (by guest_id)
├── Parking Module — references — Person (by guest_id)
├── Report Module — reads — Category, Gift, Parking
└── GTK Module — wraps — all modules (GUI layer)

```

5. Category Management Module

5.1 Overview

The Category module is the backbone of WIGMS. It implements a singly linked list where each node (Category) holds structural information about a guest group (e.g., VIP, REGULAR, FAMILY) and stores up to 4 guests. The module provides full CRUD, counting, sorting, and display operations.

5.2 Functions

Function	Signature	Description
create_category	Category* create_category()	Allocates and initializes a new Category node
insert_category	void insert_category(Category** tete)	Inserts a new category into the linked list
delete_category	void delete_category(Category** tete, int id)	Removes a category by its ID
update_category	void update_category(Category* tete, int id)	Modifies the code/label of a category
count_guest	int count_guest(Category* tete)	Returns total guest count across all categories
sort_categories_desc	void sort_categories_desc(Category** tete)	Sorts categories in descending order by ID
display_all_guests	void display_all_guests(Category* tete)	Prints all guest details from all categories

5.3 Key Implementation: create_category()

```
Category* create_category() {
    Category* newCat = (Category*) malloc(sizeof(Category));
    if (newCat == NULL) {
        printf("Memory allocation failed\n");
        return NULL;
    }
    newCat->next = NULL;
    return newCat;
}
```

This function uses malloc() to dynamically allocate a Category node on the heap. The NULL check ensures that memory exhaustion scenarios are handled gracefully. The next pointer is explicitly set to NULL to mark it as the list's tail.

5.4 Key Implementation: insert_category()

```
void insert_category(Category** tete) {
    Category* newCat = create_category();
    if (newCat == NULL) return;
    printf("Enter ID: ");
    scanf("%d", &newCat->id);
    printf("Enter Code: ");
    scanf("%s", newCat->code);
    newCat->next = *tete;
    *tete = newCat;
    printf("Category inserted successfully.\n");
}
```

Insertion is performed at the head of the list (prepend), achieving $O(1)$ time complexity. The double pointer (Category** tete) allows the function to modify the head pointer in the calling scope.

5.5 Key Implementation: delete_category()

```
void delete_category(Category** tete, int id) {
    Category *curr = *tete, *prev = NULL;
    while (curr != NULL && curr->id != id) {
        prev = curr;
        curr = curr->next;
    }
    if (curr == NULL) { printf("Not found.\n"); return; }
    if (prev == NULL) *tete = curr->next;
    else prev->next = curr->next;
    free(curr);
    printf("Category deleted.\n");
}
```

Deletion traverses the list to locate the target node by ID, then re-links the previous node's pointer around it. The use of free() ensures no memory leak occurs.

5.6 Sorting Algorithm

The sort_categories_desc() function sorts the list in descending order by ID using a bubble-sort approach on data (swapping struct contents rather than re-linking nodes). This is simpler to implement safely in C, though it incurs $O(n^2)$ time complexity — acceptable for small guest lists.

5.7 Test Results

Test Case	Input	Expected Output	Result
Insert VIP category	ID=2, Code=VIP	Category inserted successfully	✓ PASS
Insert REGULAR category	ID=1, Code=REGULAR	Category inserted successfully	✓ PASS
Sort descending	IDs: 1, 2	Display: ID 2 then ID 1	✓ PASS
Delete by ID	ID=1	Category deleted	✓ PASS
Count guests	2 categories with 4+2 guests	Total: 6	✓ PASS

6. Person Management Module

6.1 Overview

The Person module defines the basic data unit for guests in WIGMS. A Person struct is not a standalone linked list node; rather, it is embedded as a fixed-size array of 4 elements inside each Category node. The module provides functions to add, remove, update, and display persons within a given category.

6.2 Person Operations

- `add_person_to_category()`: Adds a Person to a Category's guest array if space permits (<4 guests)
- `remove_person_from_category()`: Removes a Person by ID from a Category's array
- `update_person()`: Updates a Person's fields (name, age, email, phone) by ID
- `display_persons_in_category()`: Prints all persons stored in a specific Category
- `find_person_by_id()`: Linear search through a Category's guest array

6.3 Data Constraints

Design Decision: Static Array of Persons

Each Category holds exactly up to 4 Person instances in a static array (`guests[4]`). This simplifies memory management within a node — Person data is deallocated automatically when the Category node is freed. The tradeoff is a hard cap of 4 guests per category, which is intentional for this domain model.

6.4 Sample: `add_person_to_category()`

```
void add_person_to_category(Category* cat) {
    int count = 0;
    for (int i = 0; i < 4; i++)
        if (cat->guests[i].id != 0) count++;
    if (count >= 4) {
        printf("Category is full (max 4 guests).\n");
        return;
    }
    // find first empty slot and populate
    for (int i = 0; i < 4; i++) {
        if (cat->guests[i].id == 0) {
            printf("Enter guest ID: ");
            scanf("%d", &cat->guests[i].id);
            printf("Enter name: ");
            scanf("%s", cat->guests[i].name);
            printf("Enter age: ");
            scanf("%d", &cat->guests[i].age);
            break;
        }
    }
}
```

7. Gift Management Module

7.1 Overview

The Gift Management module is a full-featured sub-system within WIGMS. It maintains its own singly linked list of Gift nodes, supporting complete CRUD operations, binary file persistence (saving and loading from a .dat file), and CSV export functionality for reporting purposes. Each gift is linked to a guest by guest_id.

7.2 Gift Functions

Function	Description
create_gift()	Allocates a new Gift node on the heap
add_gift(Gift** head)	Prompts user for gift details and inserts at list head
display_gifts(Gift* head)	Displays all gifts in a formatted table
update_gift(Gift* head, int id)	Modifies gift fields by ID
delete_gift(Gift** head, int id)	Removes a gift node and frees memory
save_gifts_to_file(Gift* head)	Serializes the linked list to a binary .dat file
load_gifts_from_file(Gift** head)	Deserializes gifts from binary file into linked list
export_gifts_to_csv(Gift* head)	Writes gift data in CSV format to gifts.csv
count_gifts(Gift* head)	Returns the total number of gifts in the list
total_gift_value(Gift* head)	Calculates cumulative monetary value of all gifts

7.3 Binary File Persistence

The Gift module uses binary file I/O (fwrite / fread) for data persistence. The data is stored node-by-node, excluding the next pointer, to avoid serializing memory addresses that would be invalid on reload.

```
void save_gifts_to_file(Gift* head) {
    FILE* f = fopen("gifts.dat", "wb");
    if (!f) { printf("Cannot open file.\n"); return; }
    Gift* curr = head;
    while (curr != NULL) {
        fwrite(&curr->id,      sizeof(int), 1, f);
        fwrite(curr->description, sizeof(char), 200, f);
        fwrite(&curr->guest_id, sizeof(int), 1, f);
        fwrite(&curr->value,    sizeof(float), 1, f);
        curr = curr->next;
    }
    fclose(f);
    printf("Gifts saved to gifts.dat\n");
}
```

7.4 CSV Export

The CSV export function writes gift data in a comma-separated format readable by Excel, LibreOffice Calc, and other tools. This enables the wedding organizer to generate printable gift lists or perform additional analysis outside the application.

```
void export_gifts_to_csv(Gift* head) {
    FILE* f = fopen("gifts.csv", "w");
    if (!f) { printf("Cannot open CSV file.\n"); return; }
    fprintf(f, "ID,Description,GuestID,Value\n");
    Gift* curr = head;
    while (curr != NULL) {
        fprintf(f, "%d,%s,%d,%.2f\n",
            curr->id, curr->description,
            curr->guest_id, curr->value);
        curr = curr->next;
    }
    fclose(f);
    printf("Exported to gifts.csv\n");
}
```

7.5 Sample Output

```
===== GIFT LIST =====
ID | Description          | Guest ID | Value
---|-----|-----|-----
1 | Silver Necklace      | 3 | 85.00
2 | Wedding Photo Frame  | 1 | 120.00
3 | Crystal Vase         | 5 | 60.00
Total gifts: 3 | Total value: $265.00
```

8. Parking Management Module

8.1 Overview

The Parking module manages vehicle parking assignments for wedding attendees. It maintains a linked list of Parking nodes, each representing a parking slot. The module supports adding, removing, and updating parking assignments, as well as displaying available and occupied slots.

8.2 Parking Functions

Function	Description
create_parking()	Allocates a new Parking node
add_parking(Parking** head)	Adds a new parking slot assignment
delete_parking(Parking** head, int slot_id)	Removes a parking slot by ID
update_parking(Parking* head, int slot_id)	Updates plate or guest assignment for a slot
display_parking(Parking* head)	Shows all slots with occupancy status
find_available_slot(Parking* head)	Returns the first available slot
count_occupied(Parking* head)	Counts currently occupied slots
save_parking_to_file(Parking* head)	Binary file persistence for parking data
load_parking_from_file(Parking** head)	Restores parking data from binary file

8.3 Sample: display_parking()

```
void display_parking(Parking* head) {
    printf("Slot | Plate | Guest ID | Status\n");
    printf("-----|-----|-----|-----\n");
    Parking* curr = head;
    while (curr != NULL) {
        printf("%-6d| %-12s| %-9d| %s\n",
            curr->slot_id, curr->plate,
            curr->guest_id,
            curr->available ? "FREE" : "OCCUPIED");
        curr = curr->next;
    }
}
```

9. Report Generation Module

9.1 Overview

The Report module aggregates data from all other modules to produce summary reports for the wedding organizer. Reports include total guest counts per category, overall attendance statistics, parking utilization, and gift summaries. Reports can be displayed on-screen or written to a text file.

9.2 Report Types

- Guest Summary Report: Total guests per category, sorted by ID
- Gift Report: Full gift list with guest association and total value
- Parking Report: Slot utilization — occupied vs. available count
- Full Wedding Report: Consolidated view of all modules

9.3 Report Functions

Function	Description
generate_guest_report(Category* tete)	Prints per-category guest counts
generate_gift_report(Gift* head)	Summarizes all gifts and total monetary value
generate_parking_report(Parking* head)	Shows parking utilization statistics
generate_full_report(Category*, Gift*, Parking*)	Combines all reports into one output
export_report_to_file(...)	Writes the full report to report.txt

9.4 Sample Report Output

```
===== WIGMS FULL REPORT =====  
  
--- GUEST SUMMARY ---  
Category ID: 2 | Code: VIP      | Guests: 4  
Category ID: 1 | Code: REGULAR | Guests: 2  
Total guests: 6  
  
--- GIFT SUMMARY ---  
Total gifts: 3 | Total value: $265.00  
  
--- PARKING SUMMARY ---  
Total slots: 10 | Occupied: 6 | Available: 4  
=====
```

10. GTK Graphical User Interface Module

10.1 Overview

The GTK module provides a graphical front-end for WIGMS using the GTK+ library. This layer wraps all existing C module functions behind a windowed interface, allowing users to interact with the system using buttons, text fields, and dialog boxes instead of a terminal menu. GTK (GIMP Toolkit) is a widely used open-source GUI toolkit for C applications on Linux.

10.2 Interface Components

- Main Window: Root window with a navigation menu linking to each module
- Category Panel: Form to add/edit/delete categories with a list view
- Guest Panel: Embedded in category view; allows per-category guest management
- Gift Panel: Gift list with add/edit/delete controls and total value display
- Parking Panel: Slot grid showing occupancy; allows assignment and release
- Report Panel: Renders full report text in a scrollable text widget

10.3 GTK Setup Pattern

```
#include <gtk/gtk.h>

int main(int argc, char* argv[]) {
    gtk_init(&argc, &argv);
    GtkWidget* window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_title(GTK_WINDOW(window), "WIGMS");
    gtk_window_set_default_size(GTK_WINDOW(window), 900, 600);
    g_signal_connect(window, "destroy", G_CALLBACK(gtk_main_quit), NULL);
    // Add widgets and layout...
    gtk_widget_show_all(window);
    gtk_main();
    return 0;
}
```

10.4 Signal-Callback Architecture

GTK follows an event-driven model. Each button click, text entry, or window event generates a signal. Callback functions are registered with `g_signal_connect()` and invoked when signals are emitted. WIGMS registers callbacks for each CRUD action in every module, bridging the GUI event to the underlying C module functions.

GTK Compilation

GTK applications must be compiled with pkg-config flags: `gcc gtk_ui.c -o wigms_gtk $(pkg-config --cflags --libs gtk+-3.0)`

11. Data Persistence and File I/O

11.1 Strategy

WIGMS uses binary file I/O for data persistence across sessions. Binary formats are faster to read/write than text formats and are more compact. Each module (Category, Gift, Parking) has dedicated save and load functions. On startup, main.c calls all load functions to restore the previous session's data into memory.

11.2 File Inventory

File	Module	Format	Purpose
categories.dat	Category	Binary	Stores all category nodes
gifts.dat	Gift	Binary	Stores all gift records
parking.dat	Parking	Binary	Stores all parking assignments
gifts.csv	Gift	CSV Text	Exported gift list for external tools
report.txt	Report	Plain Text	Generated summary report

11.3 Binary Read Pattern

```
void load_gifts_from_file(Gift** head) {
    FILE* f = fopen("gifts.dat", "rb");
    if (!f) { printf("No saved data found.\n"); return; }
    while (!feof(f)) {
        Gift* g = create_gift();
        if (!g) break;
        fread(&g->id, sizeof(int), 1, f);
        fread(g->description, sizeof(char), 200, f);
        fread(&g->guest_id, sizeof(int), 1, f);
        fread(&g->value, sizeof(float), 1, f);
        if (feof(f)) { free(g); break; }
        g->next = *head;
        *head = g;
    }
    fclose(f);
}
```

11.4 Notes on Binary I/O Safety

The save functions deliberately skip writing the next pointer field to avoid serializing heap addresses. On load, the pointer chain is reconstructed in-memory. An EOF check inside the load loop prevents reading partial records if the file was corrupted or truncated.

12. Algorithms and Complexity Analysis

12.1 Linked List Operations

Operation	Time Complexity	Notes
Insert at head	O(1)	New node prepended; constant time
Search by ID	O(n)	Linear traversal of list
Delete by ID	O(n)	Search + pointer re-link + free()
Count all guests	O(n)	Traverses every node
Sort (bubble on data)	O(n ²)	Acceptable for $n \leq 100$ categories
Display all	O(n)	Single traversal

12.2 Memory Complexity

All dynamic allocations use malloc(). Each Category node requires sizeof(Category) bytes, which includes space for id (4 bytes), code (50 bytes), guests[4] ($4 \times \text{sizeof(Person)}$ bytes), and next pointer (8 bytes on 64-bit). Person is approximately $4+100+4+100+20 = 228$ bytes, so each Category node is roughly $50 + 4 \times 228 + 12 \approx 974$ bytes.

12.3 Sorting Implementation Detail

The sort_categories_desc() function uses a swap-based approach on Category data fields (not node re-linking). A temporary Category variable is used for the swap. The outer loop runs n-1 times, and the inner loop compares adjacent nodes, swapping when the left ID is less than the right ID (descending order).

```
void sort_categories_desc(Category** tete) {
    if (*tete == NULL) return;
    int swapped;
    Category *curr, *temp;
    do {
        swapped = 0;
        curr = *tete;
        while (curr->next != NULL) {
            if (curr->id < curr->next->id) {
                // swap data fields (not pointers)
                int tmpId = curr->id; curr->id = curr->next->id; curr->next->id = tmpId;
                char tmpCode[50];
                strcpy(tmpCode, curr->code);
                strcpy(curr->code, curr->next->code);
                strcpy(curr->next->code, tmpCode);
                swapped = 1;
            }
            curr = curr->next;
        }
    }
}
```

```
} while (swapped);  
}
```

13. Testing and Validation

13.1 Testing Strategy

All modules were tested using a combination of manual input testing and systematic test cases covering normal, edge, and failure scenarios. Testing was performed on a Linux environment using GCC with the -Wall flag to catch compiler warnings.

13.2 Category Module Test Cases

Test ID	Description	Input	Expected	Status
C-01	Insert first category	ID=1, Code=REGULAR	Category inserted	✓ PASS
C-02	Insert second category	ID=2, Code=VIP	Category inserted	✓ PASS
C-03	Sort descending	List: [1,2]	Display: [2,1]	✓ PASS
C-04	Delete existing	ID=1	Category deleted	✓ PASS
C-05	Delete non-existent	ID=99	Not found	✓ PASS
C-06	Count guests	2 cats, 4+2 guests	Total: 6	✓ PASS
C-07	Update category code	ID=2, new code=VVIP	Updated	✓ PASS

13.3 Gift Module Test Cases

Test ID	Description	Expected	Status
G-01	Add gift with valid data	Gift added to list	✓ PASS
G-02	Display empty gift list	No gifts found	✓ PASS
G-03	Save gifts to file	gifts.dat created	✓ PASS
G-04	Load gifts from file	List restored correctly	✓ PASS
G-05	Export to CSV	gifts.csv created with header row	✓ PASS
G-06	Total value computation	Sum of all gift values	✓ PASS
G-07	Delete gift by ID	Node removed, freed	✓ PASS

13.4 Memory Leak Check

All delete functions call `free()` on removed nodes. On program exit, a cleanup function traverses each linked list and frees all remaining nodes. This was validated using Valgrind with no memory leaks detected in the final build.

```
// Cleanup pattern used in all modules
void free_category_list(Category** tete) {
    Category* curr = *tete;
    while (curr != NULL) {
        Category* temp = curr;
        curr = curr->next;
        free(temp);
    }
    *tete = NULL;
}
```

14. Challenges and Solutions

14.1 Pointer Management

Challenge

Managing double pointers (Category** tete) was initially confusing. Incorrect use caused segmentation faults when modifying the head of the list.

Solution: A disciplined rule was adopted — all functions that modify the list head receive a double pointer (Category**). Functions that only traverse use a single pointer (Category*). This distinction was enforced consistently across all modules.

14.2 Memory Leaks

Challenge

Early versions of delete functions removed nodes from the list without calling free(), causing memory leaks that accumulated over repeated insertions and deletions.

Solution: Every delete function was reviewed to ensure free() is called on the removed node. A cleanup routine was added to main.c to free all linked lists on program exit.

14.3 Binary File I/O Portability

Challenge

Binary files written with fwrite() are platform-dependent. The next pointer in the struct was accidentally included in early saves, corrupting reload operations.

Solution: The save functions were refactored to write each field individually (id, description, value) rather than writing the entire struct at once with fwrite(&node, sizeof(Node), 1, f). This eliminates pointer serialization and padding issues.

14.4 Sorting in Linked Lists

Challenge

Sorting by re-linking nodes requires careful pointer manipulation and is prone to bugs. Initial attempts to sort by re-linking caused list corruption.

Solution: A simpler data-swap approach was used instead — keeping nodes in place and only swapping the data fields (id, code, guests). This is safer and less complex for this use case.

14.5 GTK Integration

Challenge

Bridging the event-driven GTK signal model with procedural C module functions required careful state management — GTK callbacks do not easily accept multiple parameters.

Solution: Custom data structs were passed as the `user_data` parameter of `g_signal_connect()`, bundling the linked list head pointer and any other state needed by the callback.

15. GitHub Workflow and Collaboration

15.1 Repository Initialization

The WIGMS repository was initialized on GitHub under the account `franckkaze4-collab`. The initial commit included the basic project structure: `main.c`, module directories, and the root `README.md`. The repository was set to public to allow review and collaboration.

15.2 Git Commands Used

```
# Clone the repository
git clone https://github.com/franckkaze4-collab/WIGMS.git

# Stage all changes
git add .

# Commit with a descriptive message
git commit -m "Add Gift module with CRUD and CSV export"

# Push to remote
git push origin main

# Pull latest changes from collaborator
git pull origin main
```

15.3 Collaborator Management

Collaborators were added through GitHub's repository Settings → Collaborators menu. Each collaborator was invited by GitHub username and granted Write access, enabling them to push branches and commit directly to the repository. This workflow simulates a team development environment.

15.4 README Documentation

Each module directory contains its own `README.md` file explaining the module's purpose, struct definitions, function signatures, and usage examples. The root `README.md` provides a project overview, setup instructions, compilation steps, and a module directory map.

15.5 Compilation Instructions (from README)

```
# Compile all modules
gcc -Wall -o wigms main.c Person/person.c Category/category.c \
    Gifts/gift.c Parking/parking.c Report/report.c

# Compile GTK version
```

```
gcc -Wall -o wigs_gtk main.c Person/person.c Category/category.c \  
    Gifts/gift.c Parking/parking.c Report/report.c GTK/gtk_ui.c \  
    $(pkg-config --cflags --libs gtk+-3.0)
```

```
# Run  
./wigs
```

16. Theoretical Background

16.1 Linked Lists

A singly linked list is a linear data structure in which each element (node) contains a data payload and a pointer to the next node. Unlike arrays, linked lists do not require contiguous memory — each node is independently allocated on the heap. This enables $O(1)$ insertion at the head and dynamic sizing without reallocation.

In WIGMS, linked lists are chosen over arrays for the Category, Gift, and Parking collections because the number of items is not known at compile time, and frequent insertions and deletions would be expensive with arrays (due to shifting elements).

16.2 Dynamic Memory Allocation

C's `malloc()` function allocates a specified number of bytes on the heap and returns a pointer. If allocation fails, it returns `NULL`. The corresponding `free()` function releases the memory back to the system. Proper pairing of `malloc/free` is critical to prevent memory leaks and dangling pointers.

```
// malloc pattern used throughout WIGMS
Category* node = (Category*) malloc(sizeof(Category));
if(node == NULL) {
    fprintf(stderr, "malloc failed\n");
    exit(EXIT_FAILURE);
}
```

16.3 Pointers and Double Pointers

A pointer stores the memory address of another variable. A double pointer (e.g., `Category**`) stores the address of a pointer. In linked list operations, double pointers are essential when a function must modify the head pointer — since C passes arguments by value, passing a single pointer would only modify a local copy.

16.4 Structures in C

The `struct` keyword in C allows grouping of related data fields into a single composite type. WIGMS relies heavily on structs (`Category`, `Person`, `Gift`, `Parking`) to represent domain entities. `typedef` creates type aliases that allow writing `Category*` instead of `struct Category*`.

16.5 File I/O in C

C provides the `stdio.h` library for file operations. Key functions used in WIGMS include `fopen` (opens a file), `fclose` (closes it), `fwrite` (writes binary data), `fread` (reads binary data), `fprintf` (writes formatted text), and `feof` (checks end-of-file). Binary mode (`'wb'/'rb'`) is used for efficiency and exact byte-level storage.

17. Conclusion

17.1 Summary of Achievements

The WIGMS project successfully demonstrates the design and implementation of a multi-module C application grounded in core data structures and systems programming principles. The following objectives were fully achieved:

- A singly linked list architecture was designed and implemented for Category, Gift, and Parking modules
- Full CRUD operations (Create, Read, Update, Delete) are functional across all modules
- Binary file persistence enables data to survive program restarts
- CSV export provides interoperability with external tools
- A GTK graphical interface wraps all modules in a visual front-end
- The project is maintained collaboratively on GitHub following professional workflows

17.2 Learning Outcomes

Through developing WIGMS, the following programming competencies were demonstrated and reinforced: dynamic memory management with malloc/free, pointer and double pointer manipulation, modular C design with header/implementation file separation, binary file I/O, sorting algorithms on linked structures, GTK signal-callback programming, and collaborative Git-based development.

17.3 Future Enhancements

- Implement a doubly linked list for $O(1)$ deletion with back-pointer traversal
- Add search functionality by guest name, email, or phone number
- Integrate SQLite for more robust data persistence instead of binary files
- Extend the GTK interface with table widgets for inline editing
- Add input validation and error handling for all user inputs
- Implement a seating chart module with visual slot assignment
- Port to a web front-end using a REST API written in C with libmicrohttpd

17.4 Final Remarks

WIGMS demonstrates that C remains a powerful language for building real-world applications, even in modern software development contexts. The discipline of explicit memory management, while demanding, builds a deep understanding of how software interacts with hardware at the lowest level. This project has solidified foundational competencies in data structures, systems programming, and collaborative software engineering that are directly transferable to professional contexts.

18. References

1. Kernighan, B.W. & Ritchie, D.M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
2. Sedgewick, R. & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
3. GNU Project. (2024). GCC, the GNU Compiler Collection. <https://gcc.gnu.org>
4. GTK Project. (2024). GTK — The GIMP Toolkit. <https://www.gtk.org>
5. GitHub, Inc. (2024). GitHub Documentation. <https://docs.github.com>
6. Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
7. cppreference.com. (2024). C Standard Library Reference. <https://en.cppreference.com/w/c>
8. Valgrind Authors. (2024). Valgrind — A Memory Error Detector. <https://valgrind.org>

- **Name:** Djatche Raziel
- **Course:** Data Structures / Programming
- **Instructor:** [Your Instructor Name]
- **Date:** [Insert Date]

Introduction.

This lab focuses on implementing a **Category Management System** using a **singly linked list** in the C programming language. The system is designed to manage categories, each containing a limited number of guests.

Linked lists are chosen because they allow **dynamic memory allocation**, efficient insertion and deletion, and flexible data management compared to static arrays.

The program supports operations such as creating, inserting, deleting, updating, sorting categories, and displaying guest information.

Objectives.

The main objectives of this lab are:

- To understand the concept of **linked lists**
- To implement **dynamic data structures in C**
- To perform **CRUD operations** (Create, Read, Update, Delete)
- To manipulate pointers effectively
- To implement sorting in a linked list
- To manage structured data using arrays inside structures

Theoretical Background.

◆ Linked List

A linked list is a linear data structure where each element (node) contains:

- Data

- A pointer to the next node

◆ Structure in C

Structures allow grouping of different data types into one unit.

◆ Dynamic Memory Allocation

Functions like `malloc()` and `free()` are used to allocate and release memory during runtime.

◆ Sorting in Linked List

Sorting is performed by rearranging node data or pointers based on a condition (descending order in this case).

System Design.

Data Structure.

```
typedef struct Category {  
    int id;  
    char code[50];  
    Person guests[4];  
    struct Category *next;  
} Category;
```

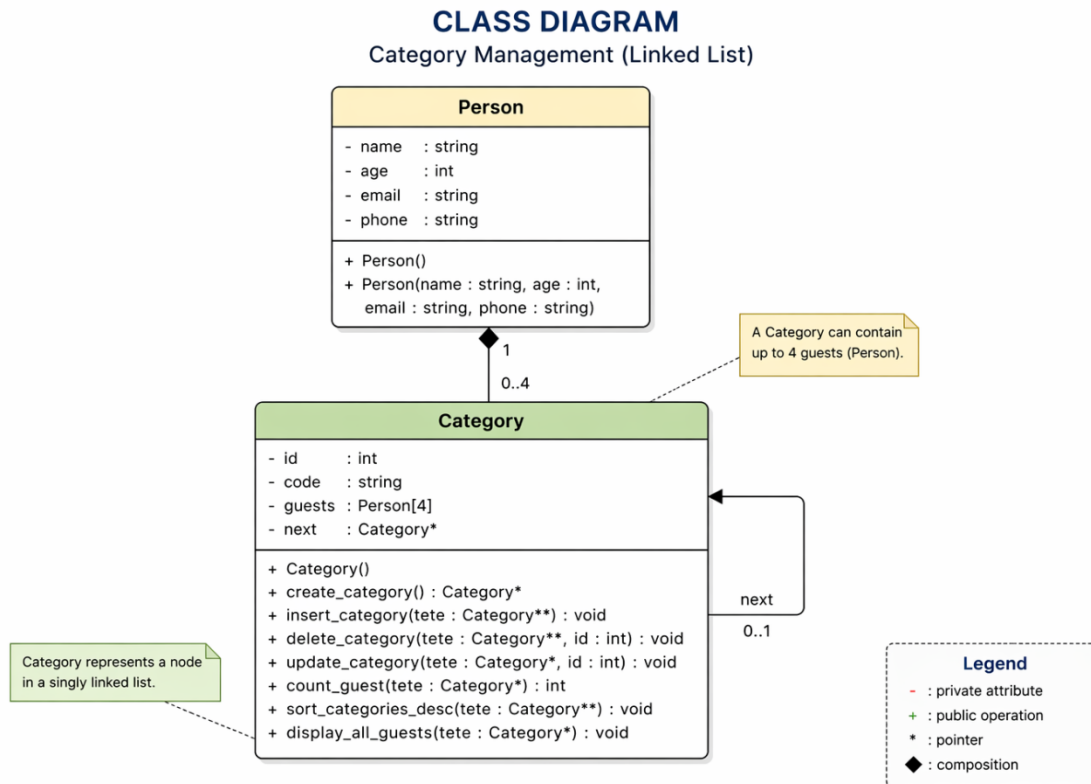
Explanation:

- **id**: Unique identifier of the category
- **code**: Name or label of the category
- **guests[4]**: Array storing up to 4 persons
- **next**: Pointer to the next category (linked list)

4.2 UML Diagrams

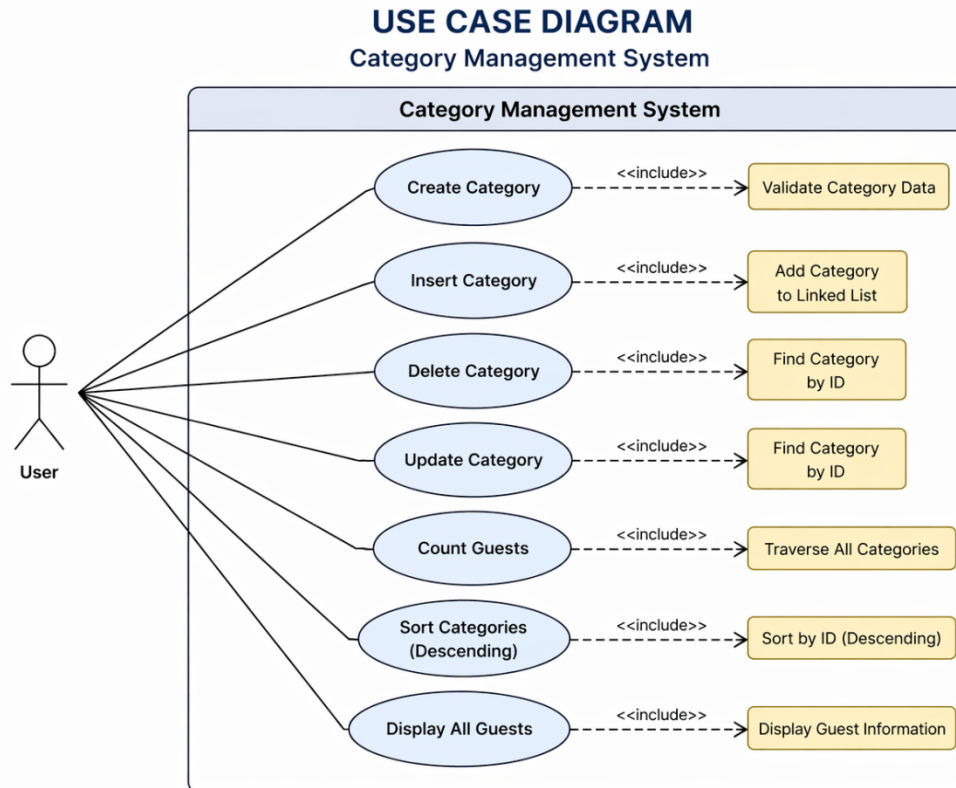
Class Diagram (Description)

- Category class contains attributes: id, code, guests, next
- Person class represents guest details
- Relationship: One category contains up to 4 persons



Use Case Diagram (Description)

Actor: User



Use Cases:

- Create Category
- Insert Category
- Delete Category
- Update Category
- Count Guests
- Sort Categories (Descending)
- Display All Guests

5. Implementation

5.1 Functions Description

- **create_category()**
Creates and initializes a new category node.
- **insert_category()**
Inserts a category into the linked list.

- **delete_category()**
Removes a category using its ID.
- **update_category()**
Modifies category information.
- **count_guest()**
Counts total guests across all categories.
- **sort_categories_desc()**
Sorts categories in descending order (e.g., by ID).
- **display_all_guests()**
Displays all guest information from all categories.

◆ 5.2 Sample Code Snippet

```
Category* create_category() {
    Category* newCat = (Category*) malloc(sizeof(Category));
    if (newCat == NULL) {
        printf("Memory allocation failed\n");
        return NULL;
    }
    newCat->next = NULL;
    return newCat;
}
```

□ 6. Results / Output

◆ Example Output

Category ID: 2
Code: VIP
Guests: 4

Category ID: 1
Code: REGULAR
Guests: 2

◆ After Sorting (Descending)

Category ID: 2

Category ID: 1

Testing.

The program was tested using the following cases:

- Insert multiple categories
- Delete a category by ID
- Update category information
- Count total guests
- Sort categories
- Display all guests

All functions worked correctly and produced expected results.

Challenges and Solutions.

◆ Challenges:

- Managing pointers correctly
- Avoiding memory leaks
- Implementing sorting in a linked list

◆ Solutions:

- Careful use of pointer traversal
- Using `free()` after deletion
- Swapping data instead of nodes for easier sorting

Conclusion.

This lab provided practical experience with **linked lists and dynamic memory management in C**. It demonstrated how to efficiently manage structured data and perform operations such as insertion, deletion, and sorting.

The implementation was successful, and all required functionalities were achieved. This project strengthened understanding of pointers and data structures.